LAPORAN TUGAS
REGRESI GRADIENT BOOSTING

Prediksi Kuat Tekan Beton – Mata Kuliah Machine Learning Pertemuan 10

Kelas : TI-C5 | Kelompok : 5

Anggota Kelompok:

1. Muhammad Irfan Nuha (434241079)
2. Bunga Lala Sal Sabila (434241066)

Tugas Kelompok

---

1. Pendahuluan

Regresi adalah teknik *supervised learning* untuk memprediksi nilai berupa **angka** (kontinu). Berbeda dengan klasifikasi yang menebak kategori, regresi menebak besaran.

Pada tugas ini kami memakai dataset **Concrete Compressive Strength** (UCI Machine Learning Repository, 1.030 sampel). Setiap baris adalah satu campuran beton beserta komposisinya dalam kg/m³ (semen, slag tanur tinggi, fly ash, air, superplasticizer, agregat kasar, agregat halus), umur beton dalam hari, dan **kuat tekan** beton dalam MPa sebagai target. Memprediksi kuat tekan penting dalam teknik sipil: pengujian laboratorium butuh waktu hingga 28 hari, sedangkan model regresi bisa memperkirakannya langsung dari resep campuran.

Metode yang kami pakai adalah **Gradient Boosting Regressor**. Sebagai pembanding, kami juga melatih **Linear Regression** untuk menunjukkan mengapa metode non-linear lebih cocok untuk data ini.

2. Metode Gradient Boosting dan Metrik Evaluasi

Gradient Boosting Regressor adalah metode *ensemble* yang menggabungkan banyak *decision tree* kecil secara **berurutan**. Pohon pertama membuat tebakan awal, lalu setiap pohon berikutnya dilatih khusus untuk memperbaiki **sisa kesalahan** (residu) dari pohon-pohon sebelumnya. Hasil akhirnya adalah jumlah kontribusi seluruh pohon. Parameter utamanya:

- **n_estimators = 100**: jumlah pohon yang dibangun.

- **learning_rate = 0,1**: seberapa besar tiap pohon boleh mengoreksi; nilai kecil membuat belajar lebih hati-hati.

- **max_depth = 3**: kedalaman maksimal tiap pohon, supaya tiap pohon tetap sederhana.

Karena dibangun dari pohon keputusan, metode ini mampu menangkap hubungan **tidak linear** dan interaksi antar-fitur, tidak perlu normalisasi data, dan tahan terhadap outlier.

Model dievaluasi dengan **MAE** (rata-rata selisih absolut, dalam MPa), **MSE** (rata-rata kuadrat selisih), **RMSE** (akar MSE, kembali ke MPa), dan **R^2** (proporsi variasi target yang dijelaskan model, 0 sampai 1, makin mendekati 1 makin baik).

3. Import Library, Input Data, dan Preprocessing

py regresi_kelompok5.py

```python
import os
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
OUT_DIR = os.path.join(BASE_DIR, "output")
os.makedirs(OUT_DIR, exist_ok=True)
TARGET = "concrete_compressive_strength"
```

Kode 1. Import library dan konfigurasi

py regresi_kelompok5.py

```python
def evaluasi(y_asli, y_pred):
    """Hitung MAE, MSE, RMSE, dan R2."""
    mse = mean_squared_error(y_asli, y_pred)
    return {"MAE": mean_absolute_error(y_asli, y_pred), "MSE": mse,
            "RMSE": np.sqrt(mse), "R2": r2_score(y_asli, y_pred)}
```

Kode 2. Fungsi evaluasi (MAE, MSE, RMSE, R²)

py regresi_kelompok5.py

```python
    # ---------------- 1. INPUT ----------------
    judul("1. INPUT DATA")
    df = pd.read_csv(os.path.join(BASE_DIR, "concrete_data.csv"))
    df.columns = df.columns.str.strip()          # ada spasi di nama kolom 'fine_aggregate '
    print(f"Ukuran data : {df.shape[0]} baris x {df.shape[1]} kolom")
    print(f"Target      : {TARGET} (MPa)")
    print(df.head().to_string(index=False))

    # ---------------- 2. PREPROCESSING ----------------
    judul("2. PREPROCESSING")
    print(f">> Missing value : {int(df.isna().sum().sum())} sel")
    dup = int(df.duplicated().sum())
    print(f">> Duplikasi     : {dup} baris")
    df = df.drop_duplicates().reset_index(drop=True)
    print(f"   -> duplikat dihapus, sisa {len(df)} baris")

    print(">> Outlier (IQR):")
    for c in df.columns:
        q1, q3 = df[c].quantile(0.25), df[c].quantile(0.75)
        iqr = q3 - q1
        n = int(((df[c] < q1 - 1.5 * iqr) | (df[c] > q3 + 1.5 * iqr)).sum())
        if n:
            print(f"     {c:<30}: {n} baris")
    print("   -> tidak dibuang: nilainya wajar untuk campuran beton (mis. umur")
    print("      beton 180-365 hari), dan Gradient Boosting tahan terhadap outlier")
```

Kode 3. Input data dan preprocessing

```
Windows PowerShell
PS D:\KULIAH\...\Pertemuan 10 - Regresi> python regresi_kelompok5.py

============================================================
  1. INPUT DATA
============================================================
Ukuran data : 1030 baris x 9 kolom
Target      : concrete_compressive_strength (MPa)
 cement  blast_furnace_slag  fly_ash  water  superplasticizer  coarse_aggregate  fine_aggregate  age  concrete_compressive_strength
  540.0                 0.0      0.0  162.0               2.5            1040.0           676.0   28                         79.99
  540.0                 0.0      0.0  162.0               2.5            1055.0           676.0   28                         61.89
  332.5               142.5      0.0  228.0               0.0             932.0           594.0  270                         40.27
  332.5               142.5      0.0  228.0               0.0             932.0           594.0  365                         41.05
  198.6               132.4      0.0  192.0               0.0             978.4           825.5  360                         44.30

============================================================
  2. PREPROCESSING
============================================================
>> Missing value : 0 sel
>> Duplikasi     : 25 baris
   -> duplikat dihapus, sisa 1005 baris
>> Outlier (IQR):
     blast_furnace_slag           : 2 baris
     water                        : 15 baris
     superplasticizer             : 10 baris
     fine_aggregate               : 5 baris
     age                          : 59 baris
     concrete_compressive_strength : 8 baris
   -> tidak dibuang: nilainya wajar untuk campuran beton (mis. umur
      beton 180-365 hari), dan Gradient Boosting tahan terhadap outlier
```

Output input data dan preprocessing

Dataset tidak memiliki missing value, tetapi terdapat **25 baris duplikat**, yaitu resep campuran yang sama persis tercatat lebih dari sekali. Duplikat dihapus agar tidak membuat model terlalu "hafal" data tertentu, sehingga tersisa **1,005** sampel.

Pengecekan IQR menemukan outlier di beberapa kolom, terbanyak pada *age* (umur beton 180 sampai 365 hari). Outlier **tidak dibuang**: nilai-nilai itu adalah resep dan umur pengujian yang sah, bukan salah input. Selain itu Gradient Boosting berbasis pohon sehingga tidak terpengaruh nilai ekstrem seperti metode berbasis rata-rata.

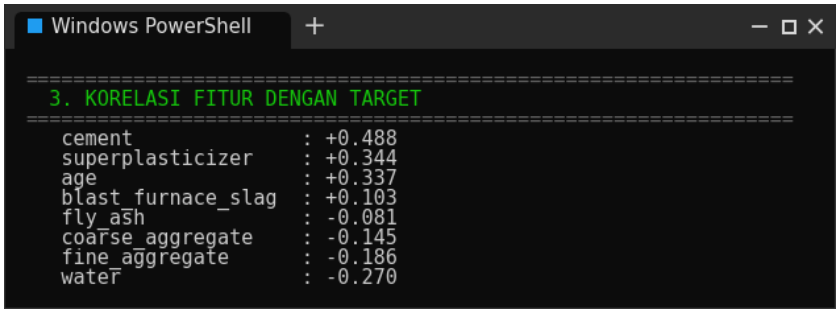
4. Eksplorasi Korelasi

py regresi_kelompok5.py

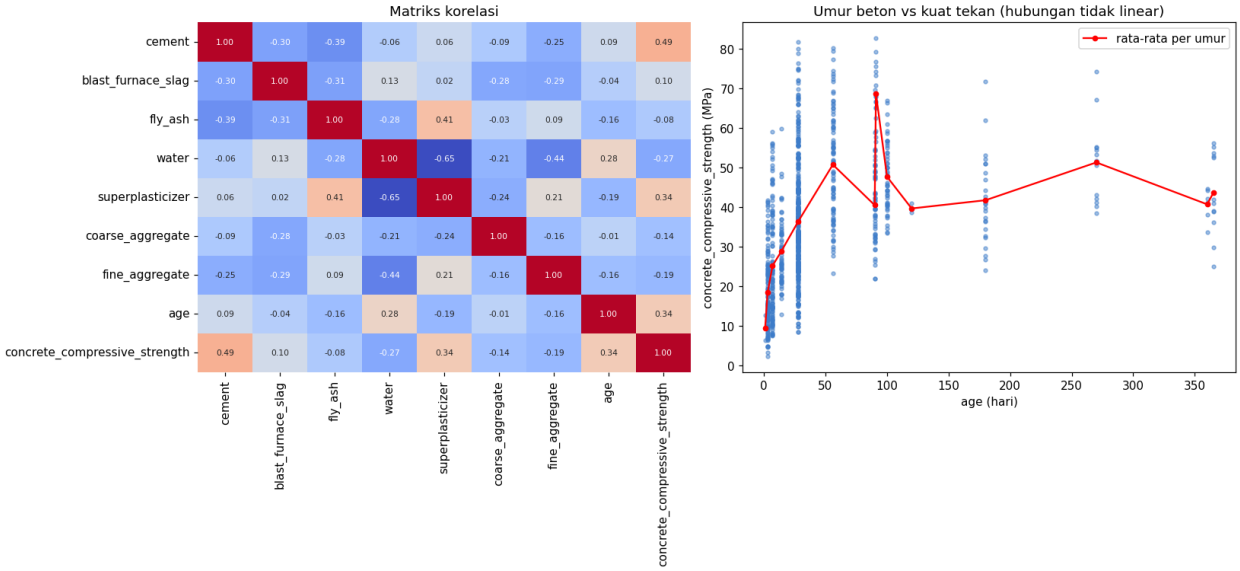
```python
# ---------------- 3. EKSPLORASI ----------------
judul("3. KORELASI FITUR DENGAN TARGET")
korelasi = df.corr()[TARGET].drop(TARGET).sort_values(ascending=False)
for f, r in korelasi.items():
    print(f"   {f:<20}: {r:+.3f}")

fig, ax = plt.subplots(1, 2, figsize=(15, 7))
sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap="coolwarm", ax=ax[0],
            annot_kws={"size": 7}, cbar=False)
ax[0].set_title("Matriks korelasi")
ax[1].scatter(df["age"], df[TARGET], s=10, alpha=0.5, color="#3d7cc9")
rata_umur = df.groupby("age")[TARGET].mean()
ax[1].plot(rata_umur.index, rata_umur.values, "r-o", ms=4, label="rata-rata per umur")
ax[1].set_xlabel("age (hari)")
ax[1].set_ylabel(f"{TARGET} (MPa)")
ax[1].set_title("Umur beton vs kuat tekan (hubungan tidak linear)")
ax[1].legend()
plt.tight_layout()
tampilkan("reg_1_eksplorasi.png")
```

Kode 4. Korelasi dan visualisasi



Output korelasi fitur dengan target



Matriks korelasi (kiri) dan umur beton vs kuat tekan (kanan)

Fitur dengan korelasi positif terbesar adalah **cement** (+0,488): makin banyak semen, makin kuat betonnya. **Water** berkorelasi negatif (-0,270): air berlebih membuat beton berpori dan lebih lemah. Yang menarik adalah **age**. Korelasinya hanya +0,337, tetapi grafik di kanan memperlihatkan kuat tekan naik tajam di hari-hari awal lalu mendatar setelah sekitar 28 hari. Pola melengkung seperti ini tidak bisa ditangkap oleh garis lurus, sehingga korelasi linear terlihat kecil padahal pengaruhnya besar. Inilah alasan kami memilih metode non-linear.

5. Split Data dan Pemodelan

py regresi_kelompok5.py

```
# ---------------- 4. SPLIT DATA ----------------
judul("4. SPLIT DATA 80% TRAINING / 20% TESTING")
X = df.drop(columns=[TARGET])
y = df[TARGET]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
print(f"   Training : {len(X_train)} baris")
print(f"   Testing  : {len(X_test)} baris")

# ---------------- 5. PEMODELAN ----------------
judul("5. PEMODELAN")
linear = LinearRegression().fit(X_train, y_train)
print("   Linear Regression (pembanding) selesai dilatih")
gbr = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, max_depth=3,
                                random_state=42)
gbr.fit(X_train, y_train)
print("   Gradient Boosting Regressor selesai dilatih")
print("     n_estimators=100 | learning_rate=0.1 | max_depth=3")
```

Kode 5. Split 80/20 dan pelatihan model

Windows PowerShell

```
============================================================
  4. SPLIT DATA 80% TRAINING / 20% TESTING
============================================================
   Training : 804 baris
   Testing  : 201 baris

============================================================
  5. PEMODELAN
============================================================
   Linear Regression (pembanding) selesai dilatih
   Gradient Boosting Regressor selesai dilatih
     n_estimators=100 | learning_rate=0.1 | max_depth=3
```

Output split data dan pemodelan

Data dibagi **80% training (804 sampel)** dan **20% testing (201 sampel)**. Kedua model dilatih pada data training yang sama: *LinearRegression()* sebagai pembanding dan *GradientBoostingRegressor()* dengan parameter yang dijelaskan di bagian 2. Fungsi *evaluasi()* menghitung keempat metrik sekaligus agar kedua model dinilai dengan cara yang sama.

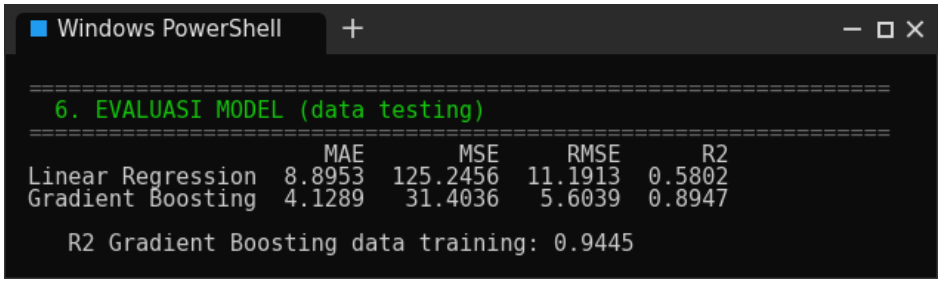
6. Evaluasi Model

py regresi_kelompok5.py

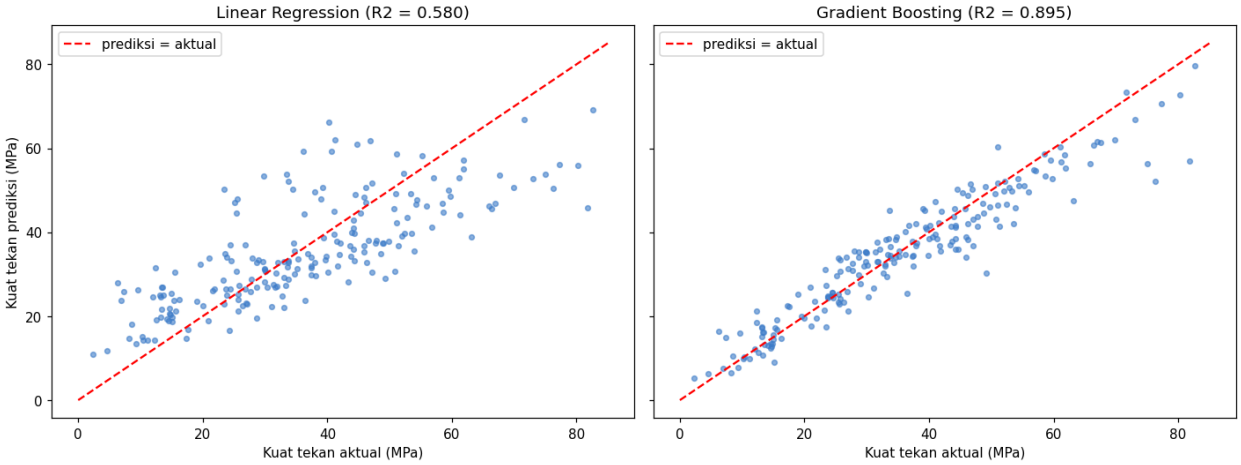
```
# ---------------- 6. EVALUASI ----------------
judul("6. EVALUASI MODEL (data testing)")
hasil = {
    "Linear Regression": evaluasi(y_test, linear.predict(X_test)),
    "Gradient Boosting": evaluasi(y_test, gbr.predict(X_test)),
}
tabel = pd.DataFrame(hasil).T
print(tabel.round(4).to_string())
r2_train = r2_score(y_train, gbr.predict(X_train))
print(f"\n   R2 Gradient Boosting data training: {r2_train:.4f}")
tabel.to_csv(os.path.join(OUT_DIR, "evaluasi.csv"))

pred_lin, pred_gbr = linear.predict(X_test), gbr.predict(X_test)
fig, ax = plt.subplots(1, 2, figsize=(13, 5), sharey=True)
for a, p, nama in [(ax[0], pred_lin, "Linear Regression"), (ax[1], pred_gbr, "Gradient Boosting")]:
    a.scatter(y_test, p, s=14, alpha=0.6, color="#3d7cc9")
    a.plot([0, 85], [0, 85], "r--", label="prediksi = aktual")
    a.set_xlabel("Kuat tekan aktual (MPa)")
    a.set_title(f"{nama} (R2 = {r2_score(y_test, p):.3f})")
    a.legend()
ax[0].set_ylabel("Kuat tekan prediksi (MPa)")
plt.tight_layout()
tampilkan("reg_2_aktual_vs_prediksi.png")
```

Kode 6. Evaluasi dan grafik aktual vs prediksi



Output evaluasi model



Aktual vs prediksi: Linear Regression (kiri) dan Gradient Boosting (kanan)

Gradient Boosting jauh mengungguli Linear Regression:

- **R^2 = 0,8947** (Linear Regression hanya 0,5802): Gradient Boosting menjelaskan sekitar 89% variasi kuat tekan beton.

- **MAE = 4,13 MPa** (Linear 8,90 MPa): rata-rata tebakan meleset sekitar 4 MPa, kurang dari setengah kesalahan Linear Regression.

- **RMSE = 5,60 MPa** (Linear 11,19 MPa).

Pada grafik, titik-titik Gradient Boosting berkumpul rapat di sekitar garis merah, sedangkan Linear Regression menyebar lebar karena tidak mampu mengikuti pola melengkung umur beton.

R^2 data training (0,9445) sedikit lebih tinggi dari data testing (0,8947). Selisih sekitar 5 poin ini wajar untuk model berbasis pohon dan masih dalam batas aman, artinya model tidak *overfitting* berlebihan dan tetap bekerja baik pada data baru.

7. Feature Importance

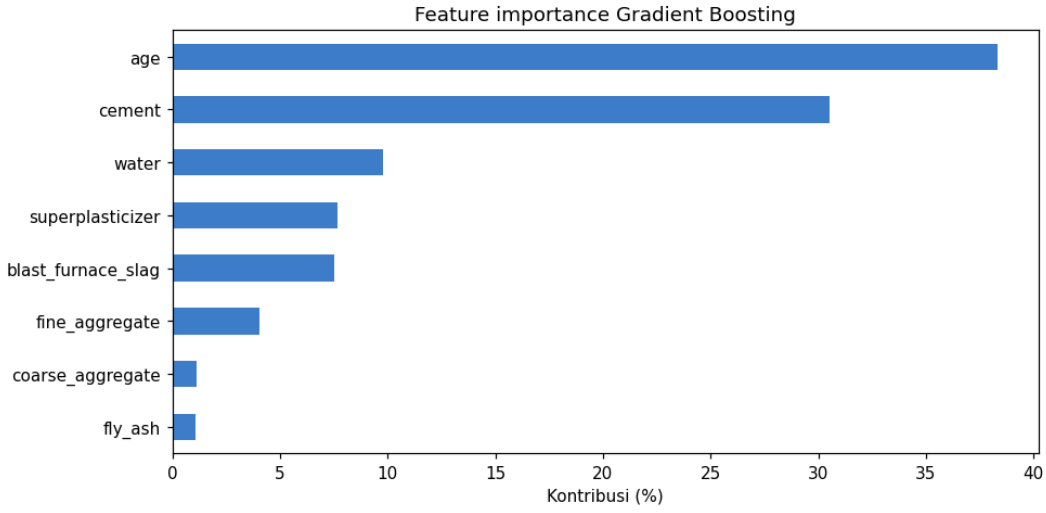
```python
# ---------------- 7. FEATURE IMPORTANCE ----------------
judul("7. FEATURE IMPORTANCE GRADIENT BOOSTING")
penting = pd.Series(gbr.feature_importances_, index=X.columns).sort_values(ascending=False)
for f, v in penting.items():
    print(f"   {f:<20}: {v * 100:5.1f}%")

plt.figure(figsize=(9, 4.5))
penting.iloc[::-1].mul(100).plot(kind="barh", color="#3d7cc9")
plt.xlabel("Kontribusi (%)")
plt.title("Feature importance Gradient Boosting")
plt.tight_layout()
tampilkan("reg_3_feature_importance.png")
```

Kode 7. Feature importance Gradient Boosting

```
Windows PowerShell
============================================================
  7. FEATURE IMPORTANCE GRADIENT BOOSTING
============================================================
   age                  :  38.3%
   cement               :  30.5%
   water                :   9.8%
   superplasticizer     :   7.7%
   blast_furnace_slag   :   7.5%
   fine_aggregate       :   4.0%
   coarse_aggregate     :   1.1%
   fly_ash              :   1.1%
```

Output feature importance



Kontribusi tiap fitur

Feature importance menunjukkan seberapa besar setiap fitur dipakai pohon-pohon Gradient Boosting untuk mengurangi kesalahan. Dua fitur mendominasi: **age (38,3%)** dan **cement (30,5%)**, disusul water (9,8%). Hasil ini sesuai teori beton: kuat tekan ditentukan oleh proses hidrasi semen yang berlangsung seiring waktu, serta perbandingan air dan semen. Perhatikan bahwa age menjadi fitur terpenting meskipun korelasi linearnya kecil, karena Gradient Boosting mampu membaca pola tidak linearnya.

8. Contoh Prediksi

py regresi_kelompok5.py

```
158     # ---------------- 8. CONTOH PREDIKSI ----------------
159     judul("8. CONTOH PREDIKSI (5 data testing)")
160     contoh = X_test.head(5)[["cement", "water", "age"]].copy()
161     contoh["Aktual"] = y_test.head(5).values
162     contoh["Prediksi"] = pred_gbr[:5].round(2)
163     contoh["Selisih"] = (contoh["Aktual"] - contoh["Prediksi"]).round(2)
164     print(contoh.to_string(index=False))
165
166     campuran = pd.DataFrame([{"cement": 350, "blast_furnace_slag": 100, "fly_ash": 0, "water": 180,
167                               "superplasticizer": 8, "coarse_aggregate": 950,
168                               "fine_aggregate": 750, "age": 28}])[X.columns]
169     print("\n   Campuran baru (semen 350, slag 100, air 180, superplasticizer 8, umur 28 hari)")
170     print(f"   Prediksi kuat tekan: {gbr.predict(campuran)[0]:.2f} MPa")
171
172     X_test.assign(Aktual=y_test.values, Prediksi=pred_gbr.round(2)).to_csv(
173         os.path.join(OUT_DIR, "hasil_prediksi.csv"), index=False)
174     print("\nHasil disimpan -> output/hasil_prediksi.csv, output/evaluasi.csv")
```

Kode 8. Contoh prediksi data testing dan campuran baru

Windows PowerShell

```
============================================================
  8. CONTOH PREDIKSI (5 data testing)
============================================================
 cement  water  age  Aktual  Prediksi  Selisih
  152.7  178.1   28   19.01     25.11    -6.10
  122.6  203.5   28   24.29     24.77    -0.48
  153.0  192.0   90   26.32     23.25     3.07
  480.0  192.0    7   34.57     38.18    -3.61
  500.1  200.0   28   44.13     45.70    -1.57

   Campuran baru (semen 350, slag 100, air 180, superplasticizer 8, umur 28 hari)
   Prediksi kuat tekan: 47.82 MPa

Hasil disimpan -> output/hasil_prediksi.csv, output/evaluasi.csv
```

Output contoh prediksi

Lima contoh data testing memperlihatkan tebakan yang dekat dengan nilai aslinya, dengan selisih antara 0,5 dan 6 MPa. Model juga bisa dipakai untuk **resep campuran baru**: campuran dengan semen 350, slag 100, air 180, dan superplasticizer 8 kg/m³ pada umur 28 hari diprediksi memiliki kuat tekan **47,82 MPa**.

9. Kesimpulan

Gradient Boosting Regressor berhasil memprediksi kuat tekan beton dengan **R^2 0,8947**, **MAE 4,13 MPa**, dan **RMSE 5,60 MPa**, jauh lebih baik dari Linear Regression (R^2 0,5802). Keunggulan ini muncul karena hubungan komposisi dan umur beton terhadap kuat tekan bersifat tidak linear. Faktor paling berpengaruh adalah umur beton dan jumlah semen, sesuai dengan teori material beton.